A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Aryan Sirohi	102303832
2	Ritvik Bajaj	102303821

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY (DEEMED TO BE UNIVERSITY)

PATIALA - 147004 (MAY -2026)

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Introduction to Problem Statement 2

2 Literature Review **4**

3 Research Gaps **7**

4 Problem Formulation **8**

5 Objectives **9**

6 Methodology **10**

7 Datasets **14**

8 Results and Discussion **15**

References **17**

List of Figures

1 **speedup in large graph for 1 thread** 16

2 **speedup in large graph for 4 threads** 16

List of Tables

1 Summary of Literature Review 4

1 Introduction

1.1 Background

The exponential growth in data-driven applications and network-centric systems has made graph algorithms a cornerstone of modern computing. From routing protocols in communication networks to financial fraud detection and road navigation systems, shortest-path algorithms play a fundamental role in solving real-world optimization problems. Among these, the Bellman-Ford algorithm holds a unique and important position , it is one of the few shortest-path algorithms capable of correctly handling graphs with negative edge weights, making it indispensable in domains such as currency arbitrage detection, network delay modelling, and constraint satisfaction systems. Despite its correctness and versatility, Bellman-Ford carries a significant computational burden. With a time complexity of O(V * E) where V is the number of vertices and E is the number of edges â the algorithm becomes extremely slow on large-scale graphs, which are increasingly common in today's applications. A social network with millions of users, a logistics network with thousands of routes, or a telecommunications grid with complex interdependencies can each contain tens of millions of edges, rendering a sequential Bellman-Ford implementation impractical. This project directly addresses this challenge. By leveraging parallel computing paradigms, specifically OpenMP, MPI, and a Hybrid (OpenMP + MPI) approach, this work demonstrates how the inherent parallelism in the Bellman-Ford algorithm can be exploited to dramatically reduce execution time and improve scalability â all without the use of a GPU, using only CPU-based parallel processing.

1.2 Introduction to Problem Statement

The algorithm operates in two phases. In the first phase, it performs V minus 1 iterations, where in each iteration all E edges are relaxed , meaning the algorithm checks whether the known shortest distance to the destination vertex of each edge can be improved by going through the source vertex of that edge. In the second phase, it performs one more pass over all edges to detect any negative weight cycle. The critical observation that makes Bellman-Ford amenable to parallelization is this: within a single iteration, all edge relaxations are independent of each other. No edge relaxation in iteration k depends on the result of another edge relaxation in the same iteration k. This data-independence is the foundation of the parallel approach explored in this project. Parallel computing has emerged as the dominant paradigm for high-performance computation. Two major models are widely used in practice:

(1.) Shared Memory Parallelism (OpenMP): Multiple threads execute concurrently and share a common memory space. OpenMP achieves this through compiler directives in C/C++ and Fortran. It is easy to implement and highly effective on multi-core processors.
(2.) Distributed Memory Parallelism (MPI as a Message Passing Interface): Multiple processes each have their own private memory. They communicate explicitly by passing messages. MPI is the standard for large-scale cluster and supercomputer environments.

(3.) Hybrid Parallelism (OpenMP + MPI): A combination of both models, where MPI manages communication between distributed processes and OpenMP handles thread-level parallelism within each process. This mirrors the architecture of real-world HPC (High Performance Computing) clusters.

Theoretical performance of parallel programs is bounded by Amdahl's Law, which states that speedup is limited by the fraction of the program that cannot be parallelised. For Bellman-Ford, the sequential portion includes the outer iteration loop control and synchronisation between iterations, while the in-

ner edge-relaxation loop is the parallelisable component. Understanding this boundary is central to the analysis in this project.

2 Literature Review

Table 1: Summary of Literature Review

References	Concepts Highlighted	Technique(s) used	Significance/ Proposal	Limitations
[1]	Parallelisation of graph traversal and shortest-path algorithms in distributed memory environments; edge partitioning strategies; communication minimisation between distributed processes	MPI-based distributed edge partitioning for efficient memory use' emotions and detecting student engagement in real-time	Demonstrated that distributing edge lists across MPI processes significantly reduces per-process computation time	overhead analysis was not conducted for small-scale simulations on a single machine

[2]	**Shared-memory parallelism of the Bellman-Ford inner edge-relaxation loop; impact of thread count on speedup; race condition handling using atomic operations**	**OpenMP parallel for directives on the edge relaxation loop; atomic operations to prevent write conflicts on shared distance array**	**Confirmed that the inner edge-relaxation loop of Bellman-Ford is highly parallelisable with near-linear speedup up to 4â6 threads**	**Did not evaluate performance on dense graphs; experiments were limited to uniform random graphs and did not test real-world network datasets; hybrid parallelism was not explored**

[3]	**Hybrid parallel computing combining shared-memory threads and distributed-memory processes; two-level parallelism architecture**	**Hybrid OpenMP + MPI implementation where MPI partitions the graph across processes and OpenMP threads handle per-process edge relaxation**	**Established that the hybrid model consistently outperforms pure OpenMP and pure MPI**	**Experiments were conducted on dedicated HPC clusters with high-speed interconnects; results may not directly translate to a single-machine simulation using VirtualBox, where inter-process communication uses shared memory**

3 Research Gaps

Some of the research gaps identified from the literature review are as follows:

- ***Absence of Small-Scale and Single-Machine Evaluation***: A recurring limitation across all three studies is that experiments were conducted on large dedicated HPC clusters, supercomputers, or multi-node environments with high-speed network interconnects. None of the studies evaluated parallel Bellman-Ford performance in a resource-constrained, single-machine setting ,such as a multi-core laptop or desktop running a virtualized Linux environment. This leaves an important practical gap, as many students, researchers, and practitioners do not have access to HPC infrastructure. This project directly addresses this by demonstrating that meaningful parallel speedup can be achieved on a standard consumer-grade CPU using Oracle VirtualBox with Ubuntu, making the findings more accessible and reproducible.

- ***Absence of Amdahl's Law analysis in Empirical Results***: Although some studies referenced Amdahl's Law theoretically, none of the three studies overlaid the theoretical speedup curve predicted by Amdahl's Law directly onto their empirical speedup graphs. This omission makes it harder to understand how far the actual implementation falls from the theoretical maximum and why. This project incorporates this analysis explicitly, plotting measured speedup alongside Amdahl's Law predictions to identify and explain the sources of performance loss â including communication overhead, memory bandwidth constraints, and load imbalance.

4 Problem Formulation

The Bellman-Ford algorithm is a well-known method used in computer science to find the shortest path between points in a network, even when some connections have negative values or costs. It is widely used in real-world applications such as network routing protocols, financial systems, and transportation networks. However, there is a serious problem. When the network becomes very large , for example, a system with thousands of nodes and tens of thousands of connections , the Bellman-Ford algorithm becomes extremely slow when run on a single processor. This is because it checks every single connection repeatedly, one after another, in a sequential manner. The time it takes grows very quickly as the network gets bigger, making it practically unusable for large real-world problems. The good news is that many of these checks are completely independent of each other meaning they do not need to wait for one another to finish. This opens the door to parallel computing, where multiple tasks are performed simultaneously, just like how multiple workers building a house finish faster than one worker doing everything alone. This project tackles this exact problem. The goal is to parallelize the Bellman Ford algorithm using three powerful technologies like OpenMP, which allows multiple threads to work simultaneously on the same computer; MPI (Message Passing Interface), which allows multiple processes to work together by passing information between them; and a Hybrid approach combining both OpenMP and MPI together for maximum performance. These are compared against the traditional sequential (single threaded) version to measure exactly how much faster parallel computing makes the algorithm. By doing this, the project demonstrates how modern parallel computing techniques can transform a slow, impractical algorithm into a fast and scalable solution , all running on a standard computer without any specialized hardware like a GPU.

5 Objectives

- **To implement a correct and optimised sequential Bellman-Ford algorithm in C as a baseline, incorporating early-exit optimisation and precise execution time measurement.**
- **To develop a parallel OpenMP version of the Bellman-Ford algorithm that distributes the edge relaxation workload across multiple CPU threads using shared memory, and analyse the effect of varying thread counts on performance.**
- **To develop a parallel MPI version of the Bellman-Ford algorithm that partitions the edge list across multiple independent processes and synchronises distance updates using collective communication operations.**
- **To develop a Hybrid OpenMP + MPI version that combines both shared-memory and distributed-memory parallelism, simulating the architecture of real-world High Performance Computing (HPC) clusters on a single multi-core machine.**
- **To benchmark and compare all four implementations sequential, OpenMP, MPI, and Hybrid across graphs of varying sizes (sparse and dense) using consistent and controlled experimental conditions.**
- **To measure and evaluate key performance metrics including speedup, parallel efficiency, and scalability for each parallelisation approach.**
- **To analyse the empirical results against Amdahl's Law and identify the practical sources of performance loss such as synchronisation overhead, communication cost, and load imbalance.**
- **To demonstrate that meaningful parallel performance gains can be achieved on a standard consumer-grade CPU without any GPU or dedicated HPC infrastructure, using Oracle VirtualBox running Ubuntu Linux.**

6 Methodology

Bellman-Ford variants :These algorithms are the variants of bellman ford . the sequential model serves as as the baseline against which all parallel versions are compared.

Algorithm 1 : Sequential Bellman-Ford

```
Input: Graph G(V, E), source vertex src
Initialize dist[src] = 0, dist[v] = ∞ for all v ≠ src
for iteration = 1, 2, ..., V − 1 do
    relaxed ← false
    for each edge (u, v, w) in E do
        if dist[u] + w < dist[v] then
            dist[v] ← dist[u] + w
            relaxed ← true
        end if
    end for
    if relaxed = false then break
    end if
end for
for each edge (u, v, w) in E do
    if dist[u] + w < dist[v] then
        Report: Negative Cycle Detected
    end if
end for
Output: dist[ ]
```

Algorithm 2 : Parallel Bellman-Ford using OpenMP

```
Input: Graph G(V, E), source vertex src, thread count T
Initialize dist[src] = 0, dist[v] = ∞ for all v ≠ src
for iteration = 1, 2, ..., V − 1 do
    relaxed ← false
    #pragma omp parallel for num_threads(T)
    for each edge (u, v, w) in E do
        if dist[u] + w < dist[v] then
            Atomically update dist[v] ← dist[u] + w
            relaxed ← true
        end if
    end for
    (All threads synchronise at implicit barrier)
    if relaxed = false then break
    end if
end for
Output: dist[ ]
```

Algorithm 3 : Parallel Bellman-Ford using MPI

```
Input: Graph G(V, E), source vertex src, process count P
Initialize dist[src] = 0, dist[v] = ∞ for all v ≠ src
Partition edge list E into P equal chunks, one per process
for iteration = 1, 2, ..., V − 1 do
    relaxed ← false
    for each edge (u, v, w) in local partition do
        if dist[u] + w < dist[v] then
            local_dist[v] ← dist[u] + w
            relaxed ← true
        end if
    end for
    MPI_Allreduce(local_dist → dist, operation = MIN)
    MPI_Allreduce(relaxed → global_relaxed, operation = OR)
    if global_relaxed = false then break
    end if
end for
Output: dist[ ] on root process
```

Algorithm 4 : Hybrid Bellman-Ford using OpenMP + MPI

```
Input: Graph G(V, E), source vertex src, process count P, thread count T
Initialize dist[src] = 0, dist[v] = ∞ for all v ≠ src
Partition edge list E into P equal chunks, one per MPI process
for iteration = 1, 2, ..., V − 1 do
    relaxed ← false
    #pragma omp parallel for num_threads(T)
    for each edge (u, v, w) in local partition do
        if dist[u] + w < dist[v] then
            Atomically update local_dist[v] ← dist[u] + w
            relaxed ← true
        end if
    end for
    (All threads synchronise at implicit barrier)
    MPI_Allreduce(local_dist → dist, operation = MIN)
    MPI_Allreduce(relaxed → global_relaxed, operation = OR)
    if global_relaxed = false then break
    end if
end for
Output: dist[ ] on root process
```

7 Datasets

No external datasets were used in this project. Instead, a custom graph generator program was developed in C (generategraph.c) to synthetically generate weighted directed graphs of varying sizes. The generator produces graphs with a guaranteed connected structure by first creating a spanning chain across all vertices, followed by randomly assigned additional edges with weights ranging from 1 to 100. Graphs were generated across three configurations to test both sparse and dense scenarios:

8 Results and Discussion

The results regarding the Time Complexity are as follows:

Sequential: O(V * E) : the algorithm performs V minus 1 outer iterations, and in each iteration relaxes all E edges, giving a total complexity of O(V * E).

OpenMP: O(V * (E/T)) : in each of the V minus 1 outer iterations, the E edges are divided equally among T threads, so each thread processes E/T edges per iteration.

MPI: O(V * (E/P)) + O(V * log P) : in each of the V minus 1 outer iterations, each MPI process handles E/P edges independently. After every iteration, MPI Allreduce introduces an additional communication cost of O(log P), accumulated over V iterations.

Hybrid (OpenMP + MPI): O(V * (E/(T*P))) + O(V * log P) : each MPI process receives E/P edges, which are further divided among T threads, so each thread processes E/(T*P) edges per iteration. The MPI Allreduce communication cost remains the same as pure MPI.

Where V = number of vertices, E = number of edges, T = number of OpenMP threads, and P = number of MPI processes.

Results regarding speedup for large graphs

```
vertices:5000
edges: 25000
threads:1
Terminal
running OpenMp bellman-ford from source vertex 0...
running all 4999 iterations(no early exit-benchmarking mode)
status:success(no negative cycle)

--timing results--
execution time :0.082026 seconds
execution time :82.026 milliseconds
```

Figure 1: speedup in large graph for 1 thread

```
threads:4

running OpenMp bellman-ford from source vertex 0...
running all 4999 iterations(no early exit-benchmarking mode)
status:success(no negative cycle)

--timing results--
execution time :0.046750 seconds
execution time :46.750 milliseconds
```

Figure 2: speedup in large graph for 4 threads

References

[1] G. Hajela and M. Pandey, "Parallel implementations for solving shortest path problem using bellman-ford," *Int. J. Comput. Appl*, vol. 95, no. 15, pp. 2–6, 2014.

[2] M. Arun *et al.*, "Comparative analysis of parallelised shortest path algorithms using open mp," in *2017 third international conference on sensing, signal processing and security (ICSSS)*. IEEE, 2017, pp. 369–378.

[3] N. ALHafez and A. Kurdi, "Parallel paradigms in modern hpc: A comparative analysis of mpi, openmp, and cuda," *arXiv preprint arXiv:2506.15454*, 2025.